

Functions as Values

First-Class Functions · Closures · Currying · Composition · Memoization

Interview Preparation — Basic → Intermediate → Expert

Core Concept — First-Class Functions

In JavaScript, functions are **first-class citizens**. This means a function is treated exactly like any other value — it can be **stored in a variable**, **passed as an argument** to another function, and **returned** from a function. This is the foundation of functional programming in JS and comes up in almost every senior-level JavaScript interview.

■ One-line interview answer: A first-class function means functions are values — you can assign them, pass them around, and return them just like numbers or strings.

■ BASIC — Storing & Calling Functions

1. Function Expression

What is it?

A **Function Expression** assigns an anonymous (nameless) function to a variable. Unlike a function declaration, it is **not hoisted** — you cannot call it before the line where it is defined. The variable name becomes the way you reference and call the function.

Why interviewers ask this:

They want to check if you understand the difference between a function declaration (**hoisted**, available everywhere in scope) vs a function expression (**not hoisted**, only available after assignment). This is a classic hoisting question.

```
// Function Expression vs Declaration
// Function Declaration – hoisted ■
greet(); // works even before the definition
function greet() { return 'Hello!'; }

// Function Expression – NOT hoisted ■
sayHi(); // TypeError: sayHi is not a function
const sayHi = function() { return 'Hi!'; };

// Calling a function expression
const add = function(a, b) { return a + b; };
add; // f (a, b) → it's a function reference
add(); // NaN → called but no args
add(2, 3); // 5 → called with args ■
```

■ The () is the trigger. Without () you get the function object. With () you execute it.

2. Copying a Function Reference

What is it?

Since functions are values, you can assign the same function to multiple variables. Both variables point to the **exact same function in memory** — no copy is made. This is called holding a **reference**.

```
// Copying a function reference
const greet = (name) => `Hi, ${name}!`;

const sayHello = greet; // no () → copy the reference, not call it

greet('Akshay'); // 'Hi, Akshay!'
sayHello('Akshay'); // 'Hi, Akshay!' – same function, different name

console.log(greet === sayHello); // true → they point to same function
```

3. Passing a Function as an Argument (Callback)

What is it?

A **callback** is a function passed as an argument to another function, to be **called later** — either synchronously (right away) or asynchronously (after some event, timer, or API response). This is the backbone of event handling, array methods like `.map()` and `.filter()`, and all async patterns in JavaScript.

Why interviewers ask this:

Callbacks are everywhere — `setTimeout`, event listeners, promises, array methods. Understanding them shows you understand the event loop and async flow.

```
// Callback pattern
const runOperation = (a, b, operation) => {
  return operation(a, b); // calls whatever fn is passed in
};

const add = (a, b) => a + b;
const multiply = (a, b) => a * b;

runOperation(4, 2, add); // 6 – add is the callback
runOperation(4, 2, multiply); // 8 – multiply is the callback
runOperation(4, 2, (a, b) => a - b); // 2 – inline arrow callback
```

■ INTERMEDIATE — Functions That Return Functions

4. Higher-Order Functions (HOF)

What is it?

A **Higher-Order Function** is any function that either: (1) takes one or more functions as arguments, OR (2) returns a function as its result — or both. HOFs are what make patterns like callbacks, currying, composition, and decorators possible. `.map()`, `.filter()`, `.reduce()` are all HOFs built into JS.

Why interviewers ask this:

It is a direct interview question at mid-to-senior level: 'What is a higher-order function? Give an example.' You must be able to define it clearly and write one from scratch.

```
// Higher-Order Functions
// HOF that TAKES a function
const applyTwice = (fn, value) => fn(fn(value));

const double = x => x * 2;
applyTwice(double, 3); // double(double(3)) → double(6) → 12

// HOF that RETURNS a function – Factory pattern
const createMultiplier = (factor) => {
  return (number) => number * factor; // returns a new function
};

const double2 = createMultiplier(2); // returns fn: n => n * 2
const triple = createMultiplier(3); // returns fn: n => n * 3

double2(5); // 10
triple(5); // 15
double2(10); // 20
```

■ Step by step: `createMultiplier(2)` runs → returns `(number) => number * 2` → that returned function is stored in `double2` → `double2(5)` calls it with `number=5` → 10.

5. Closures

What is it?

A **closure** is a function that **remembers the variables from its outer scope even after that outer function has finished executing**. When a function is returned from another function, it carries a backpack of all the variables it had access to when it was created. That backpack is the closure. This is how private state is achieved in JavaScript without classes.

Why interviewers ask this:

Closures are one of the most commonly tested JS concepts — from junior to senior. Questions range from 'What is a closure?' to tricky loop-closure bugs to building a counter factory. Master this one.

```
// Closure – private state via counter factory
const createCounter = (start = 0) => {
  let count = start; // private variable – lives in closure

  return { // returned object holds closures over 'count'
    increment: () => ++count,
    decrement: () => --count,
    reset: () => { count = start; },
    value: () => count,
  };
};

const counterA = createCounter(10);
const counterB = createCounter(0); // separate closure, separate state

counterA.increment(); // 11
counterA.increment(); // 12
counterB.increment(); // 1 ← completely independent

counterA.value(); // 12 – A and B never share state ■
counterB.value(); // 1
```

■ The variable 'count' is NOT accessible from outside — you cannot do counterA.count. The only way to interact with it is through the returned functions. This is the Module pattern — closures as encapsulation.

6. Immediately Chained Function Calls fn(a)(b)

What is it?

When a function **returns another function**, you can call the returned function immediately with a second set of parentheses, without storing it in a variable first. This creates the **fn(a)(b)** syntax — the result of the first call IS the function used for the second call. This is how currying looks in practice.

```
// Chained function calls
const greet = (greeting) => (name) => `${greeting}, ${name}!`;

// Two-step (store the middle result)
const sayHello = greet('Hello'); // returns (name) => `Hello, ${name}!`
sayHello('Akshay'); // 'Hello, Akshay!'

// One-step (chain immediately)
greet('Hello')('Akshay'); // 'Hello, Akshay!'
greet('Namaste')('Akshay'); // 'Namaste, Akshay!'

// Reading the chain:
// greet('Hello') → returns a function
// (that fn)('Akshay') → calls it with 'Akshay'
```

■ EXPERT — Currying · Composition · Memoization

7. Currying

What is it?

Currying is the technique of transforming a function that takes **multiple arguments** into a **sequence of functions** each taking **one argument at a time**. Instead of calling `f(a, b, c)` you call `f(a)(b)(c)`. Named after mathematician Haskell Curry. The key benefit is **partial application** — you can lock in some arguments now and supply the rest later, creating specialized reusable functions.

Why interviewers ask this:

Currying separates junior from mid/senior. Interviewers ask: 'What is currying?', 'Write a `curry()` utility function', 'What is the difference between currying and partial application?'. Also comes up in React — event handlers often use curried functions.

```
// Currying – basic
// Normal function – all args at once
const add = (a, b, c) => a + b + c;
add(1, 2, 3); // 6

// Curried version – one arg at a time
const curriedAdd = (a) => (b) => (c) => a + b + c;

curriedAdd(1)(2)(3); // 6 – all at once

const add1 = curriedAdd(1); // locks a=1, returns (b) => (c) => 1+b+c
const add1and2 = add1(2); // locks b=2, returns (c) => 1+2+c
add1and2(3); // 6
add1and2(10); // 13 – reuse with different c ■
```

```
// Currying – real world
// Real world – tax calculator
const calcPrice = (taxRate) => (discount) => (price) => {
  return price - (price * discount / 100) + (price * taxRate / 100);
};

const withGST = calcPrice(18); // tax rate locked
const gstWith100ff = withGST(10); // discount locked

gstWith100ff(1000); // 1080 → reusable for any price ■
gstWith100ff(2000); // 2160

// React example – curried event handler
const handleChange = (field) => (event) => {
  setState(prev => ({ ...prev, [field]: event.target.value }));
};

// In JSX:
// onChange={handleChange('email')} – no need for separate handlers
```

■ **Currying vs Partial Application:** Currying always produces single-argument functions one at a time. Partial application fixes SOME arguments and returns a function that accepts the REST at once. `fn.bind(null, a)` is partial application, not currying.

8. Function Composition

What is it?

Function Composition is combining two or more functions so that the **output of one becomes the input of the next**. Think of it as a data pipeline or assembly line. There are two flavors: **compose** (right to left) and **pipe** (left to right, more readable). Libraries like Lodash/FP and RxJS are built entirely on this idea.

Why interviewers ask this:

Composition shows you understand functional programming principles. Interviewers may ask you to implement a `compose()` or `pipe()` utility from scratch using `.reduce()` or `.reduceRight()`. This also tests your mastery of HOFs.

```
// Function composition – pipe vs compose
// Individual pure functions – each does ONE thing
const trim = str => str.trim();
const toLower = str => str.toLowerCase();
const removeSpaces = str => str.replace(/\s+/g, '-');
const addSlug = str => `/blog/${str}`;

// pipe: left → right (data flows in reading order)
const pipe = (...fns) => (value) =>
  fns.reduce((acc, fn) => fn(acc), value);

// compose: right → left (math notation)
const compose = (...fns) => (value) =>
  fns.reduceRight((acc, fn) => fn(acc), value);

const createSlug = pipe(trim, toLower, removeSpaces, addSlug);

createSlug(' Hello World '); // '/blog/hello-world' ■

// Without pipe – deeply nested and hard to read ■
addSlug(removeSpaces(toLower(trim(' Hello World '))));
```

■ Key principle: Every function in the pipeline should be PURE — same input always gives same output, no side effects. This makes composed pipelines predictable and easy to test.

9. Memoization

What is it?

Memoization is a performance optimization technique where you **cache the result of a function call** based on its arguments, so that if the same arguments are passed again, the cached result is returned immediately without re-computing. It uses a closure to maintain a private cache (usually an object or Map). Best applied to **pure, expensive functions** — like recursive algorithms, heavy calculations, or API result caching.

Why interviewers ask this:

Memoization combines three concepts: closures (private cache), HOFs (wrapper function), and performance optimization. It appears in system design questions, React (useMemo, useCallback), and algorithm optimization (Fibonacci, DP). Implementing memoize() from scratch is a common coding challenge.

```

// Memoization — generic utility
const memoize = (fn) => {
  const cache = new Map(); // private cache via closure

  return (...args) => {
    const key = JSON.stringify(args);

    if (cache.has(key)) {
      console.log('CACHE HIT:', key);
      return cache.get(key); // return instantly ■
    }

    const result = fn(...args);
    cache.set(key, result); // store for next time
    return result;
  };
};

// Slow Fibonacci without memoization —  $O(2^n)$ 
const fib = (n) => n <= 1 ? n : fib(n - 1) + fib(n - 2);

// Fast Fibonacci WITH memoization
const fastFib = memoize((n) => n <= 1 ? n : fastFib(n-1) + fastFib(n-2));

fastFib(10); // computes → 55
fastFib(10); // CACHE HIT → 55 instantly
fastFib(40); // very fast now vs exponential without memo ■

```

■■ React's `useMemo` and `useCallback` are memoization built into the framework. `useMemo` caches a computed value. `useCallback` caches a function reference. Both prevent unnecessary re-computation on re-renders.

10. Generic `curry()` Utility — Write It From Scratch

What is it?

The patterns above hardcode the number of levels. A **generic `curry()`** function accepts any function and returns a curried version that works regardless of how many arguments the original function takes. It uses `fn.length` (the expected argument count) and recursion to keep collecting arguments until all are supplied, then calls the original function.


```
// Generic curry utility
const curry = (fn) => {
  return function curried(...args) {
    if (args.length >= fn.length) {
      return fn(...args); // all args collected → call original
    }
    return (...moreArgs) => curried(...args, ...moreArgs); // wait for more
  };
};

// Works on ANY multi-arg function
const add = (a, b, c) => a + b + c;
const cAdd = curry(add);

cAdd(1)(2)(3); // 6 — one at a time
cAdd(1, 2)(3); // 6 — mix of batch and single
cAdd(1)(2, 3); // 6 — flexible ■
cAdd(1, 2, 3); // 6 — all at once still works
```

Interview Questions & Model Answers

Q1: What is a first-class function?

A: A function is first-class when it is treated as a value — it can be stored in a variable, passed as an argument, and returned from another function. JavaScript functions are always first-class citizens.

Q2: What is the difference between a function declaration and a function expression?

A: Function declarations are hoisted — they are available throughout the entire scope before their definition line. Function expressions are assigned to variables and are NOT hoisted — calling them before the assignment line throws a `TypeError`.

Q3: What is a higher-order function? Give an example.

A: A higher-order function either accepts a function as an argument or returns a function (or both). Built-in examples: `.map()`, `.filter()`, `.reduce()`, `setTimeout()`. Custom example: a `createMultiplier(factor)` that returns a multiplier function.

Q4: What is a closure? Where is it used in real projects?

A: A closure is a function that retains access to variables from its outer scope even after the outer function has returned. Real uses: counter factories, private state (module pattern), memoization caches, React hooks (`useState` uses closures internally), `debounce/throttle` utilities.

Q5: What is currying? Difference from partial application?

A: Currying transforms `f(a, b, c)` into `f(a)(b)(c)` — a chain of single-argument functions. Partial application fixes some arguments and returns a function expecting the REST at once. `bind()` does partial application. Currying always produces single-argument steps.

Q6: What is function composition? Implement `pipe()`.

A: Composition means chaining functions so each one's output is the next one's input. `pipe()` processes left to right. Implementation: `const pipe = (...fns) => (val) => fns.reduce((acc, fn) => fn(acc), val);`

Q7: What is memoization? When should you use it?

A: Memoization caches function results by argument signature so repeated calls with the same arguments skip recomputation. Use it for: expensive pure calculations, recursive algorithms (Fibonacci, DP), React renders (useMemo/useCallback). Don't use it for impure functions or where memory is constrained.

Q8: What does `fn(a)(b)` mean? Walk me through it.

A: `fn(a)` calls `fn` with argument `a`. If `fn` returns another function, `(b)` immediately calls THAT returned function with argument `b`. This is the chained call pattern used in currying and factory functions.

Mental Model — The () Rule

```
// The () Mental Model
fn // the function object itself (no execution)
fn() // execute it, get the return value
fn()() // execute → get returned fn → execute that too
fn(a)(b) // execute with a → get fn back → execute with b
fn(a)(b)(c) // currying – three levels

// Reading a chain left to right:
createLogger('INFO')('Server started')
// ↑ returns fn ↑ calls that fn

// Every () = one function execution
// Every argument inside () goes to that level's function
```

Pattern	Looks Like	Core Idea
Function expression	<code>const fn = () => {}</code>	Function stored as a value
Callback	<code>runIt(fn)</code>	Function passed as argument
Higher-Order Function	<code>fn</code> returns <code>fn</code>	Functions that operate on functions
Closure	Inner <code>fn</code> + outer variable	Private state via scope memory
Chained call	<code>fn(a)(b)</code>	Call the returned function immediately
Currying	<code>fn(a)(b)(c)</code>	One arg at a time, partial application
Composition / Pipe	<code>pipe(f, g, h)(x)</code>	Chain functions – output feeds next
Memoization	<code>memoize(fn)(x)</code>	Cache results via closure

■ Golden Rule: If a function returns a function — you need two sets of `()` to fully execute it. Add one `()` per level of nesting.